

Lab 5: Spatial Filtering – Smoothing and Sharpening

Student Nawaf Rabea Shahbal 2240009025

Loading the Images

```
In [50]: import os
import cv2
from skimage import data, img_as_float, exposure, io
import matplotlib.pyplot as plt
import matplotlib
import numpy as np
import skimage as ski

matplotlib.rcParams['font.size'] = 18

images = (

    'brick',
    'horse',

)

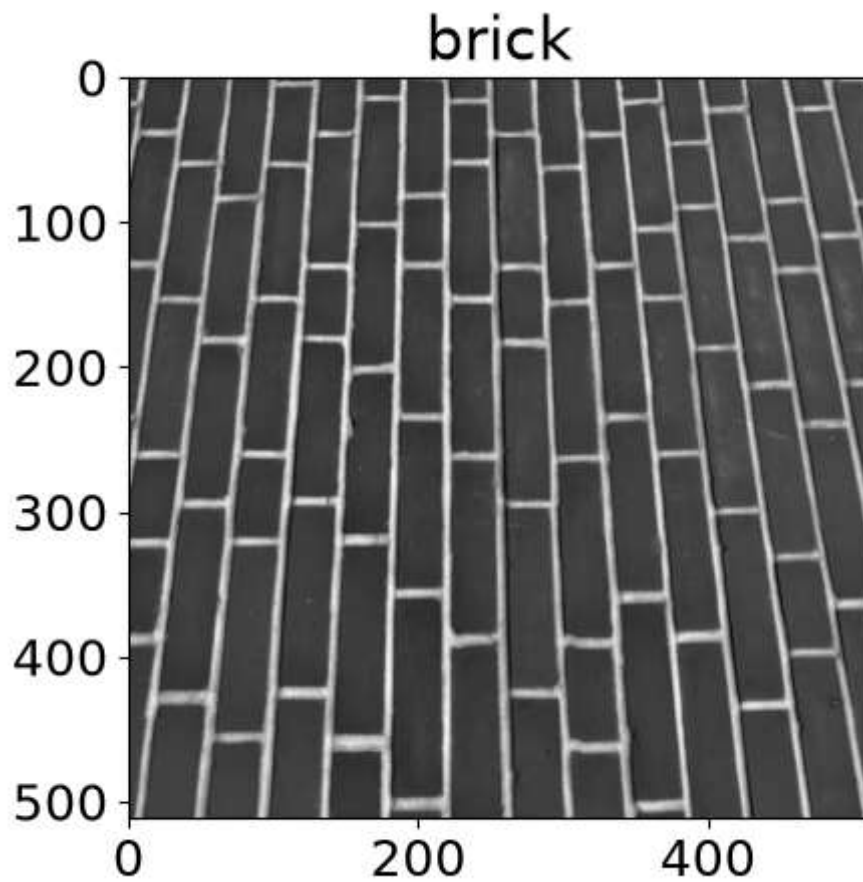
# . Define the folder name and create it if it doesn't exist
output_folder = "images"
os.makedirs(output_folder, exist_ok=True)

# . Iterate through your images and save them
for name in images: # Assuming 'images' list is defined earlier
    caller = getattr(ski.data, name)
    image = caller()

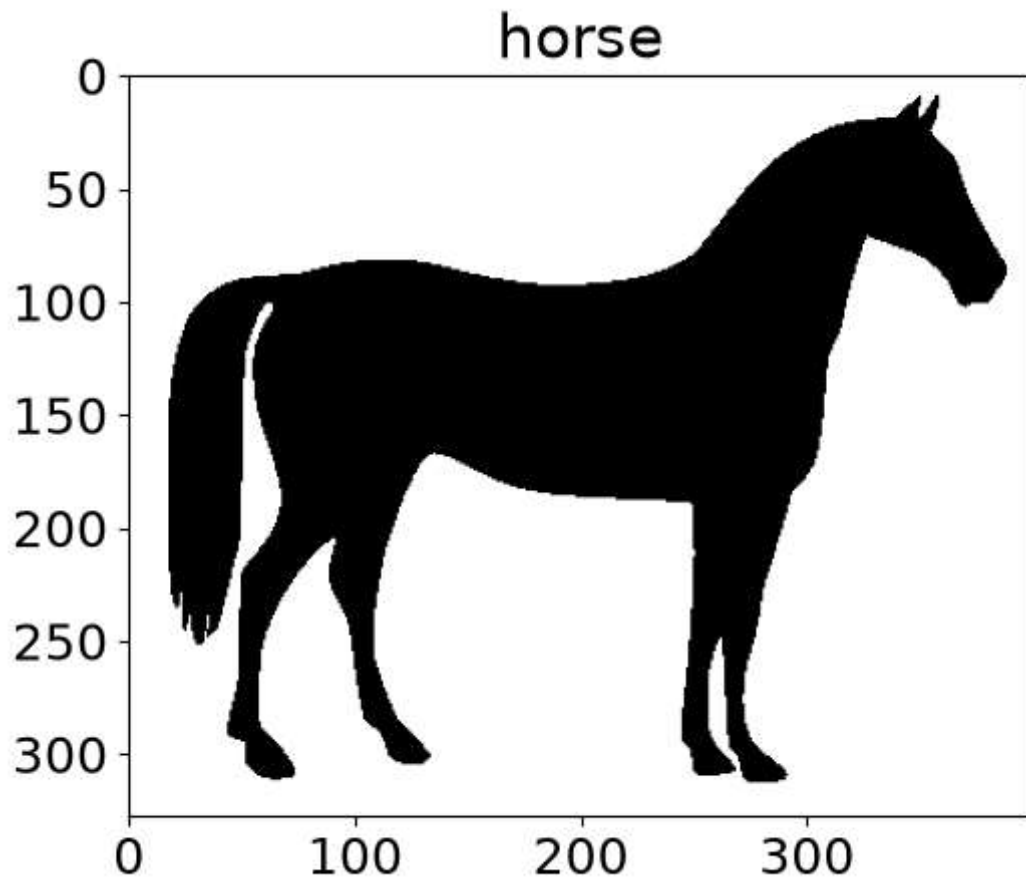
    # Save the image to the folder
    file_path = os.path.join(output_folder, f"{name}.png")
    ski.io.imsave(file_path, image)

    # Display the image as you did before
    plt.figure()
    plt.title(name)
    plt.imshow(image, cmap='gray')
    plt.show()

## Images Are Saved in the Folder Now
```



```
C:\Users\pZwZ\AppData\Local\Temp\ipykernel_19804\3519757457.py:30: UserWarning: images\horse.png is a boolean image: setting True to 255 and False to 0. To silence this warning, please convert the image using img_as_ubyte.  
    ski.io.imsave(file_path, image)
```



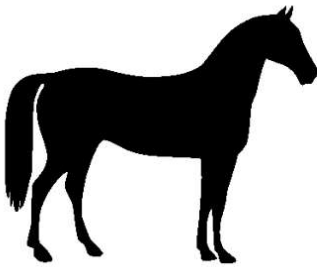
Task 1 UNSHARP

```
In [49]: image = cv2.imread('images/horse.png')
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB) # Convert from BGR to RGB
# Gaussian sigma
sigma=2
#parameter
amount=1.5
# Convert image to float32 for processing
image_float = image.astype(np.float32) / 255.0
# Apply Gaussian blur
blurred = cv2.GaussianBlur(image_float, (0, 0), sigmaX=sigma,
sigmaY=sigma)
# Perform Unsharp Masking
sharpened = np.clip(image_float + amount * (image_float - blurred), 0, 1)

# Plot original, blurred, and sharpened images
fig, ax = plt.subplots(1, 3, figsize=(15, 5))
ax[0].imshow(image)
ax[0].set_title("Original Image")
ax[0].axis("off")
ax[1].imshow(blurred)
ax[1].set_title("Blurred Image")
ax[1].axis("off")
ax[2].imshow(sharpened)
ax[2].set_title("Sharpened (Unsharp Masking)")
```

```
ax[2].axis("off")
plt.show()
```

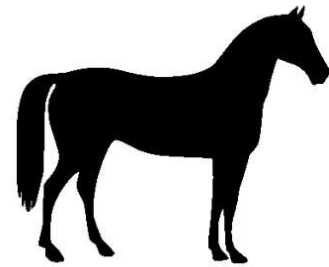
Original Image



Blurred Image



Sharpened (Unsharp Masking)



Task 2 : 7x7 blur box

```
In [46]: from IPython.core.pylabtools import figsize
from matplotlib.pyplot import axes
image2 = cv2.imread('images/brick.png') #Loading

image2 = cv2.cvtColor(image2, cv2.COLOR_BGR2RGB) #Converting to RGB

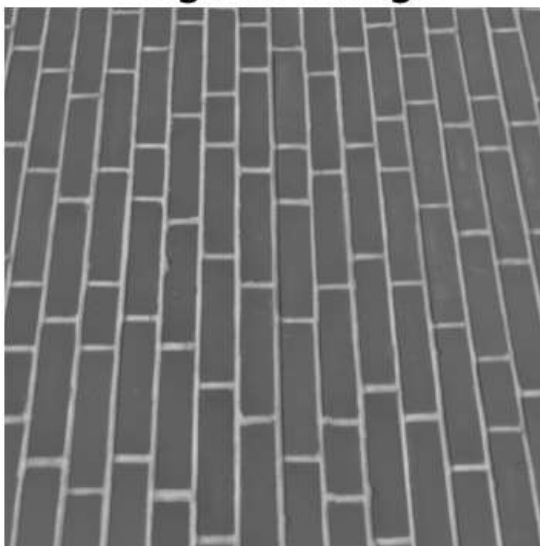
SmoothedBox = cv2.blur(image2 , (7,7))

fig, axes = plt.subplots(1,2, figsize=(10,5))

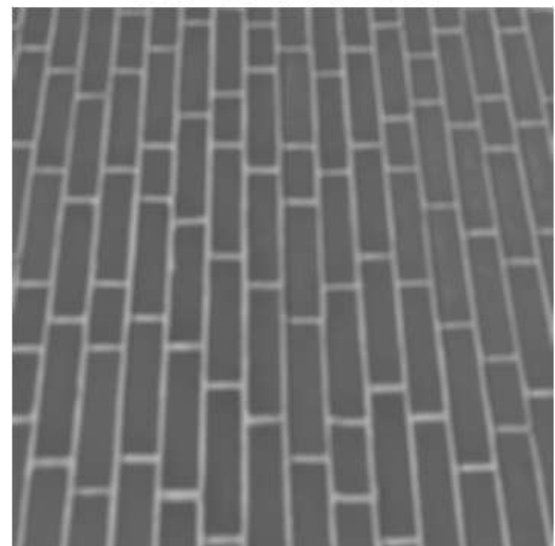
axes[0].imshow(image2)
axes[0].set_title("Original Image")
axes[0].axis('off')
axes[1].imshow(SmoothedBox)
axes[1].set_title("7x7 Box Filter")
axes[1].axis('off')
```

```
Out[46]: (np.float64(-0.5), np.float64(511.5), np.float64(511.5), np.float64(-0.5))
```

Original Image

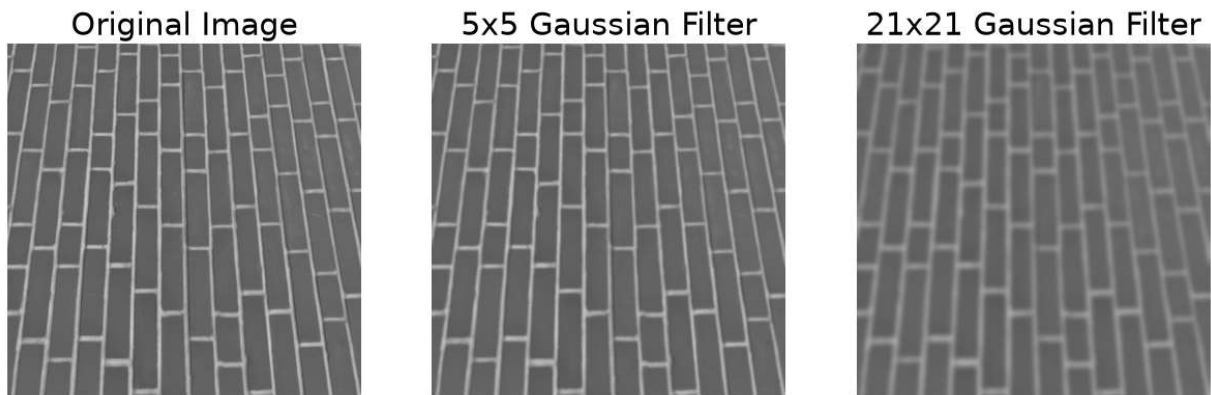


7x7 Box Filter



Task 3 : Gaussian Filter

```
In [47]: # Apply a 5x5 Gaussian Filter
gaussian_5x5 = cv2.GaussianBlur(image2, (5, 5), 0)
# Apply a 21x21 Gaussian Filter
gaussian_21x21 = cv2.GaussianBlur(image2, (21, 21), 0)
# Plot all three images side by side
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(image2)
axes[0].set_title("Original Image")
axes[0].axis('off')
axes[1].imshow(gaussian_5x5)
axes[1].set_title("5x5 Gaussian Filter")
axes[1].axis('off')
axes[2].imshow(gaussian_21x21)
axes[2].set_title("21x21 Gaussian Filter")
axes[2].axis('off')
plt.show()
```



Task 4 : Laplucian Filters (Sharp)

```
In [48]: imageFloat = np.float32(image2)/255.0

laplucian = cv2.Laplacian(imageFloat, cv2.CV_32F, ksize=3)

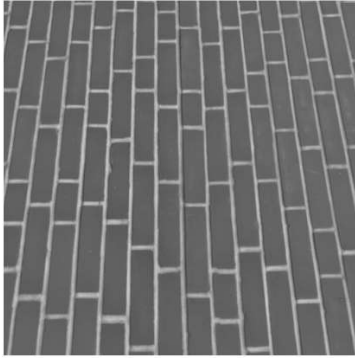
sharpenedImage = np.clip(imageFloat - laplucian, 0,1)

fig, axes = plt.subplots(1, 3, figsize=(18, 6))
axes[0].imshow(imageFloat)
axes[0].set_title("Original Image")
axes[0].axis('off')
# Note: We take the absolute value of the Laplacian just so it displays properly
# (otherwise negative edge values render weirdly in matplotlib)
axes[1].imshow(np.abs(laplucian))
axes[1].set_title("Laplacian Filter (Edges)")
axes[1].axis('off')
axes[2].imshow(sharpenedImage)
axes[2].set_title("Sharpened Image")
axes[2].axis('off')
```

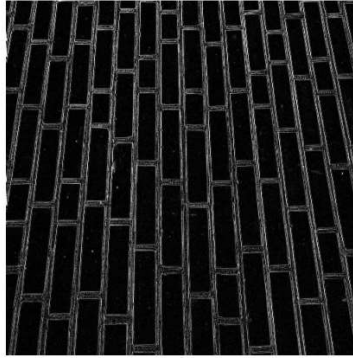
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..1.4901962].

Out[48]: (np.float64(-0.5), np.float64(511.5), np.float64(511.5), np.float64(-0.5))

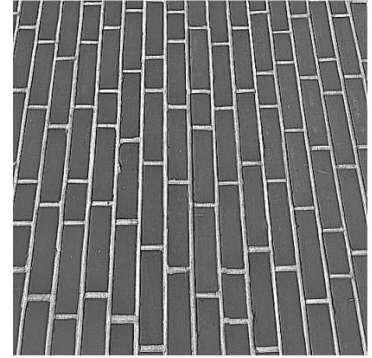
Original Image



Laplacian Filter (Edges)



Sharpened Image



In []: